DataPup: Schema-Aware and Execution-Aware Text-to-SQL for Human-in-the-Loop Analytical Database Clients

Sahith Vibudhi
Independent Researcher
San Francisco, California, USA
v.sahithkumar@gmail.com

Krishna Chaitanya Balusu
Independent Researcher
San Francisco, California, USA
krishnabkc15@gmail.com

ABSTRACT

DataPup is an open-source, AI-assisted database client that translates natural language queries into SQL for analytical databases including ClickHouse, PostgreSQL, and MySQL. Featured in the LangChain ecosystem and the official ClickHouse documentation, DataPup uses a LangChain-based agent with seven schema discovery tools to iteratively introspect database structure and generate SQL. A key challenge in production is how to present schema information to the LLM: the choice of format, scope, metadata, and examples affects query correctness and the quality of the human-in-the-loop database experience. We study these prompt dimensions on three ClickHouse benchmarks (206 queries, 10 tables) with two Claude models, then validate with Claude, Codex, and Gemini CLI runs plus a focused DuckDB experiment. On the 150-query custom analytics benchmark, DataPup's current full-schema JSON style achieves only 17.3% result correctness, while the revised prompt configuration reaches 66.0% (McNemar exact p=2.7×10^{-19}). Compared with a stronger full-schema zero-shot Markdown baseline, the revised configuration reaches 66.0% versus 55.3% RC in a planned paired comparison (p=0.015 raw; p=0.075 Holm-corrected). A leakage audit confirms that the few-shot pool is separate from the evaluation set; removing three near-duplicate flagged queries leaves best-configuration RC unchanged at 66.0% on the remaining 147 queries. In the black-box CLI setting, the best configuration remains strongest for all three providers after one execution-repair attempt, reaching 56.0% RC for Claude, 50.7% for Codex, and 53.3% for Gemini. On a focused DuckDB validation over 130 portable queries, the revised prompt matches the full-schema JSON baseline at 60.8% RC while reaching 100.0% execution success. The results motivate a DataPup production design that combines compact schema presentation, audited example retrieval, execution-aware repair, and explicit surfacing of remaining semantic assumptions to analysts.

KEYWORDS

Text-to-SQL, Large Language Models, Prompt Engineering, Schema Linking, ClickHouse, OLAP, AI-Assisted Database Client

PVLDB Reference Format:
Sahith Vibudhi and Krishna Chaitanya Balusu. DataPup: Schema-Aware and Execution-Aware Text-to-SQL for Human-in-the-Loop Analytical Database Clients. PVLDB, 19(1): XXX-XXX, 2026.
doi:XX.XX/XXX.XX

This work is licensed under the Creative Commons BY-NC-ND 4.0 International License. Visit https://creativecommons.org/licenses/by-nc-nd/4.0/ to view a copy of this license. For any use beyond those covered by this license, obtain permission by emailing info@vldb.org. Copyright is held by the owner/author(s). Publication rights licensed to the VLDB Endowment.
Proceedings of the VLDB Endowment, Vol. 19, No. 1 ISSN 2150-8097.
doi:XX.XX/XXX.XX

PVLDB Artifact Availability:
The source code, data, and experimental artifacts are available at https://github.com/DataPupOrg/DataPup-Research.

1 INTRODUCTION

AI-assisted database clients are becoming essential tools for data analysts, enabling natural language interaction with complex database systems. DataPup is an open-source, cross-platform database client that uses LLM-based agents to help analysts explore and query analytical databases through natural language. In building and deploying DataPup, we encountered a fundamental design question: *how should database schema information be presented to the LLM to maximize SQL generation accuracy?*

This question is critical for analytical databases (OLAP) like ClickHouse, DuckDB, and Snowflake, which present distinct challenges compared to the transactional databases (OLTP) that dominate existing Text-to-SQL benchmarks [22]. OLAP systems feature columnar storage optimized for aggregation queries, specialized functions for time-series analysis, SQL dialect variations, and production schemas with hundreds of columns—all of which generic prompting strategies fail to capture.

Despite recent progress in LLM-based Text-to-SQL, with frontier models achieving execution accuracy exceeding 85% on Spider [22], the impact of schema presentation on SQL generation remains underexplored. When a user asks "Show me the top 10 customers by revenue last month," the LLM must understand which tables exist, what columns are available, their data types, relationships, and database-specific syntax. The effectiveness of SQL generation depends heavily on how this schema information is communicated—yet DataPup's production system naively includes all tables in raw JSON format, a strategy we found to be suboptimal.

In this paper, we present a systematic ablation study of schema-aware prompt engineering for Text-to-SQL targeting ClickHouse, investigating four dimensions of prompt design—schema format, scope, metadata enrichment, and example selection—across three benchmarks and two models. The findings directly inform DataPup's production schema presentation strategy, closing the loop between controlled experimentation and deployed system improvement.

Our contributions include:

- The design and deployment of DataPup, an open-source AI-assisted database client with a LangChain-based agent architecture for natural language querying of analytical databases.

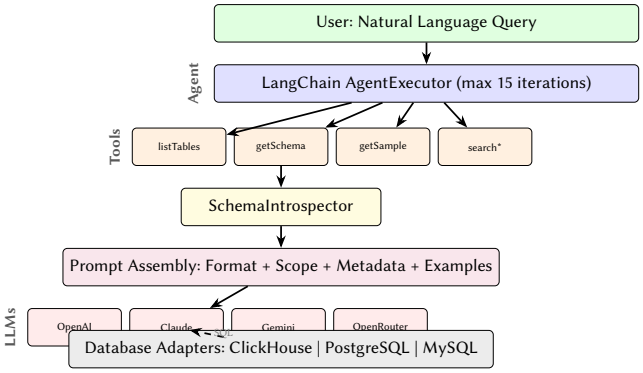

Figure 1: DataPup system architecture. The LangChain agent iteratively calls schema discovery tools, assembles prompts using the strategies evaluated in this paper, and routes to the user's selected LLM provider.

- A four-dimensional ablation analysis of schema format, scope, metadata, and example selection, with paired significance tests, a leakage audit for example retrieval, and a direct comparison against DataPup's current full-schema JSON prompt style.
- A multi-benchmark evaluation spanning 206 queries across three datasets with distinct structural characteristics (wide single-table, normalized multi-table, star schema), with cross-model validation on Claude 3.5 Sonnet and Claude Sonnet 4.
- A production-style CLI validation across Claude, Codex, and Gemini command-line agents, plus a focused DuckDB validation, including execution scoring and targeted SQL repair.
- Actionable guidelines for practitioners building AI-assisted database tools, with open-source release of our benchmark and evaluation framework.

2 SYSTEM OVERVIEW: DATAPUP

DataPup[1] is an open-source, cross-platform (macOS, Windows, Linux) AI-assisted database client that helps analysts explore and query analytical databases through natural language. The project is MIT-licensed with 294 GitHub stars and 41 forks as of early 2026, featured in the LangChain ecosystem and integrated into the official ClickHouse documentation as a recommended client. Built with Electron, React, and TypeScript, DataPup supports ClickHouse, PostgreSQL, and MySQL through a unified adapter layer. Figure 1 provides an overview of the system architecture.

2.1 Agent Architecture

DataPup's natural language query pipeline is built on LangChain's agent framework. The core agent uses an `AgentExecutor` with tool-calling capabilities, supporting up to 15 iterative tool invocations per query. The system supports four LLM providers—OpenAI, Anthropic Claude, Google Gemini, and OpenRouter—enabling users to select their preferred model.

[1]https://github.com/DataPupOrg/DataPup

The agent has access to seven structured tools for schema discovery and query analysis:

(1) `listDatabases` — enumerate available databases
(2) `listTables` — retrieve tables within a database
(3) `getTableSchema` — introspect column names, types, and constraints
(4) `getSampleRows` — fetch sample data for context
(5) `searchTables` / `searchColumns` — pattern-based schema search
(6) `analyzeQueryPerformance` — EXPLAIN-based query optimization

Each tool is defined as a `DynamicStructuredTool` with Zod schemas for type-safe argument validation. Per-session `BufferMemory` enables multi-turn conversations, and a tool results cache avoids redundant introspection calls within a session. The agent dynamically injects context—the current database and previously explored tables—into each prompt.

2.2 Schema Discovery Pipeline

The `SchemaIntrospector` module performs live schema introspection via database-native commands (e.g., `DESCRIBE TABLE`, `SHOW TABLES`). It constructs a structured representation of the database schema including column names, types, nullability, and defaults.

A critical design decision is the `getRelevantTables()` method, which determines which tables to include in the LLM prompt. In the current production system, this method returns *all* tables in the database—effectively the "Full Schema" strategy evaluated in our experiments. This naive approach motivates the present study: our ablation analysis (Section 6) shows that relevant-subset scoping can reduce prompt size substantially while preserving or directionally improving correctness, directly informing the next iteration of this component.

2.3 End-to-End Query Processing

To illustrate the system's operation, we trace a representative query: *"Show me monthly revenue trends for the past year."* The processing proceeds as follows:

(1) **Query reception.** The Electron frontend sends the natural language query and the active database connection ID to the LangChain agent via IPC.
(2) **Schema discovery.** The agent calls `listTables` to enumerate tables, then `getTableSchema` on candidates (e.g., `events`, `products`) to retrieve column definitions.
(3) **Prompt assembly.** The schema is formatted (currently raw JSON; migrating to Markdown per our findings), combined with dialect-specific guidance, and injected into the prompt template alongside any few-shot examples.
(4) **SQL generation.** The prompt is sent to the selected LLM provider. The model returns SQL using ClickHouse-specific functions (e.g., `toStartOfMonth`, `sumIf`).
(5) **Execution and display.** The generated SQL is executed against the live database through the appropriate adapter. Results are returned to the frontend for tabular display and optional visualization.

If the initial SQL fails to execute, the agent receives the error message and may iterate—calling additional schema tools or revising the query—for up to 15 rounds before returning a failure response. This iterative correction loop recovers from common errors such as incorrect column names or missing table qualifiers.

2.4 Human-in-the-Loop Interaction Model

DataPup is designed as an analyst-facing client rather than a fully autonomous query engine. Generated SQL is returned with the assistant response, executed results are shown in the database workspace, and execution errors are surfaced back into the chat so the user can refine the request. This creates three intervention points: before execution, where the analyst can inspect generated SQL; after execution, where the analyst can inspect the result shape; and after failure, where the analyst can use the error message to steer correction. The current release exposes SQL and error text, but does not yet explicitly label semantic assumptions such as filters, grouping keys, tie-breaking rules, or date windows. The repair and failure-mode experiments in this paper are therefore used to drive the next system iteration: execution repair should be automatic, while semantic uncertainty should be made visible for human confirmation.

3 BACKGROUND AND MOTIVATION

3.1 Text-to-SQL with Large Language Models

Text-to-SQL systems translate natural language queries into executable SQL. The advent of LLMs shifted the field toward prompt engineering strategies that leverage pre-trained knowledge of SQL syntax and database concepts. DAIL-SQL [6] demonstrated that careful prompt design achieves state-of-the-art results on Spider, but these studies predominantly target SQLite and PostgreSQL with schemas averaging 5–10 tables, leaving OLAP systems underexplored.

3.2 ClickHouse and OLAP Characteristics

ClickHouse is an open-source columnar database for real-time analytical queries, powering observability platforms and analytics systems at companies including Uber, Cloudflare, and eBay. It introduces several challenges for Text-to-SQL:

- Specialized aggregate functions: `argMax`, `argMin`, `groupArray`, `quantile`
- Time-series functions: `toStartOfMonth`, `toStartOfWeek`, `dateDiff`
- Array and nested data types: `Array(String)`, Nested structures
- Engine-specific behaviors: MergeTree ordering, materialized views
- Large schemas: Production deployments often exceed 100 columns per table

3.3 Research Gap

Benchmarks like Spider [22] and BIRD [10] do not capture OLAP-specific challenges. No systematic study has evaluated schema presentation strategies for analytical databases with dialect-specific syntax. Our work addresses this gap with a multi-benchmark, multi-model evaluation.

4 METHODOLOGY

We investigate four dimensions of schema-aware prompt engineering, each with multiple strategy variations. Our evaluation follows an ablation design: we establish a baseline configuration, then vary one factor at a time (OFAT) to isolate the marginal contribution of each dimension.

4.1 Schema Representation Formats

We evaluate four formats for presenting database schema information to LLMs: **Format A: CREATE TABLE (SQL DDL)** — standard SQL data definition language, including column names, types, and engine specifications. **Format B: Markdown Table** — human-readable tabular format with columns for name, type, and description. **Format C: JSON Schema** — structured JSON representation with explicit field semantics. **Format D: Natural Language** — prose descriptions of tables and columns.

4.2 Schema Scope Strategies

For databases with large schemas, including all tables and columns may exceed context limits or dilute attention. We evaluate: *Full Schema*, *Relevant Subset* (keyword-based table selection), *Progressive Expansion* (iterative schema revelation), and *User-Guided* (explicit table specification).

4.3 Metadata Enrichment

Beyond basic schema structure, additional metadata may improve generation accuracy: column descriptions, sample values, statistics, and combinations thereof.

4.4 Example Selection Methods

We compare: zero-shot (no examples), static few-shot (fixed 3 examples), dynamic few-shot (similarity-based selection), and schema-matched (examples using overlapping tables). Few-shot examples are drawn from a separate curated pool of 44 question-SQL pairs rather than from the 150-query evaluation benchmark. To check contamination risk, we audit exact natural-language overlap, exact SQL overlap, and high question-similarity pairs before interpreting the example-selection results.

4.5 Pilot Study: Initial Baseline Evaluation

Prior to the full ablation analysis, we conducted a pilot study (600 API calls) evaluating all four schema formats under default settings (full scope, no metadata, zero-shot). This pilot served two purposes: (1) identifying catastrophically failing formats to exclude from subsequent analysis, and (2) informing improvements to the evaluation pipeline.

The pilot revealed that Markdown format achieved the highest execution accuracy (92.7% EX, 30.7% RC), while natural language format completely failed (0% EX) because prose descriptions omit exact `database.table` identifiers. JSON format suffered severe degradation (48.7% EX) due to attention dilution from verbose structure (3,566 tokens vs. 1,829 for Markdown). These findings

Table 1: Custom Analytics Query Categories

Category	Count	Challenge Focus
Simple SELECT	25	Basic filtering, column selection
Aggregation	30	GROUP BY, aggregate functions
Window Functions	25	Rankings, running totals, partitions
Time-Series	30	Date functions, period comparisons
Complex JOINs	20	Multi-table reasoning, subqueries
ClickHouse-Specific	20	argMax, arrays, dialect syntax

established Markdown as the baseline format for subsequent OFAT analysis.

Critically, the pilot also revealed that result correctness is sensitive to evaluation details such as percentage normalization, scalar matching, and tolerance-based comparison. We therefore treat the format study as a pilot for choosing the representation used in later experiments. All headline prompt-configuration claims below use the final evaluator, identical 150-query set, and paired per-query comparisons.

5 EXPERIMENTAL SETUP

5.1 Benchmark Datasets

We evaluate on three datasets with distinct structural characteristics, totaling 206 queries across 10 tables.

Custom Analytics (150 queries, 4 tables). Our primary benchmark targets a ClickHouse analytics schema with `events`, `users`, `sessions`, and `products`. Table 1 shows the distribution across six complexity categories.

ClickBench (43 queries, 1 table with 105 columns). The ClickBench [4] benchmark consists of analytical queries against the `hits` table, a single wide table with 105 columns representing web analytics data. This dataset tests the model's ability to navigate very wide schemas and select appropriate columns from a large pool without multi-table JOIN reasoning.

Star Schema Benchmark (13 queries, 5-table star schema). SSB [14] provides a classical star schema with a central `lineorder` fact table and four dimension tables (`customer`, `supplier`, `part`, `date`). This tests multi-table JOIN reasoning on a normalized analytical schema with foreign key relationships.

5.2 Models and Inference Configuration

The controlled ablation study uses two frontier models from Anthropic:

- **Claude 3.5 Sonnet** (`claude-3-5-sonnet-20241022`): Our primary evaluation model, a frontier model with strong code generation capabilities and a 200K-token context window.
- **Claude Sonnet 4** (`claude-sonnet-4-20250514`): A newer-generation model used for cross-model validation to assess whether prompt engineering findings generalize across model versions.

For these API-based experiments, inference uses temperature 0.0 for deterministic output, with a maximum output length of 2,048 tokens.

To validate the findings in a more operational setting, we also run a CLI-based matrix over the full 150-query custom analytics benchmark using the Claude, Codex, and Gemini command-line assistants in noninteractive mode. These tools are intentionally treated as black-box assistants: the harness sends the same prompt text over standard input, extracts SQL from stdout, executes the SQL against the same ClickHouse instance, and optionally asks the same CLI to repair only SQL that fails execution. The CLI experiment is not a controlled leaderboard comparison because the tools do not expose identical model, temperature, or decoding controls; instead, it measures whether the DataPup prompt design remains useful across the heterogeneous assistant interfaces practitioners actually use.

5.3 Evaluation Metrics

We measure performance across multiple dimensions:

- **Execution Accuracy (EX)**: Percentage of queries that execute without syntax errors.
- **Result Correctness (RC)**: Percentage producing correct output via semantic equivalence checking with tolerance-based numeric comparison, percentage normalization, and order-insensitive matching.
- **Schema Linking Accuracy (SL)**: Correct identification of tables and columns (F1 score).
- **Token Efficiency (TE)**: Prompt tokens required per query.
- **Model Success (MS)**: Percentage of CLI calls that return non-empty SQL before timeout.
- **Repair Success (RS)**: Percentage of failed-execution SQL queries corrected by one model-guided repair attempt.

5.4 Statistical Methodology

To quantify uncertainty and support rigorous comparison, we employ the following statistical methods:

- **Bootstrap Confidence Intervals**: We compute 95% CIs via bootstrap resampling with 10,000 iterations over per-query correctness indicators. All RC values are reported with CIs in parenthetical format, e.g., 59.3 (54.2–64.1).
- **McNemar's Test**: For pairwise comparison of configurations on the same query set, we use McNemar's test on the 2×2 contingency table of per-query outcomes.
- **Holm–Bonferroni Correction**: For exploratory pairwise comparisons within an ablation dimension, we apply Holm–Bonferroni correction to control family-wise error rate at $\alpha = 0.05$. For the five headline contrasts, we report both raw and Holm-adjusted p-values.

All headline result tables are generated from saved per-query outputs and rescored with the final result comparator. The artifact bundle includes the prompt text, predicted SQL, gold SQL, execution status, result-match decision, CLI stdout/stderr logs, repair attempts, and the few-shot leakage audit.

6 RESULTS

We present findings from our ablation evaluation on the custom analytics benchmark (150 queries), followed by system prompt ablation,

Table 2: Schema Format Comparison (Full Scope, No Metadata, Zero-Shot). RC values show 95% bootstrap CIs.

Format	EX	RC	SL-F1	Tok.	Lat.
DDL	0.907	0.293 (0.22–0.37)	0.808	1,403	2,530
Markdown	0.927	0.307 (0.23–0.38)	0.836	1,829	2,614
JSON	0.487	0.173 (0.11–0.23)	0.825	3,566	2,767
Natural Lang.	0.000	0.000	0.810	1,284	2,742

Table 3: Schema Scope Comparison. RC values show 95% bootstrap CIs.

Scope	RC
Relevant Subset	0.593 (0.51–0.67)
User-Guided	0.567 (0.49–0.65)
Full	0.553 (0.47–0.63)
Progressive	0.433 (0.35–0.51)

cross-model validation, cross-provider CLI validation, DuckDB validation, execution repair, cross-dataset generalization, and external baseline comparison.

6.1 RQ1: Schema Representation Format

Markdown format achieves the highest execution accuracy (92.7%) and result correctness (30.7%), as shown in Table 2, marginally outperforming DDL (90.7% EX, 29.3% RC). McNemar's test shows this difference is not statistically significant ($p = 0.581$ for EX, $p = 0.727$ for RC). JSON format suffers a catastrophic drop to 48.7% EX despite the highest token count, indicating attention dilution from verbose structure. Natural language format achieves 0% EX—prose descriptions omit the exact `database.table` syntax required for ClickHouse execution, despite strong schema linking (0.810 F1).

6.2 RQ2: Schema Scope Strategy

Using Markdown format, we evaluate four schema scope strategies with the improved evaluation pipeline (Table 3).

Relevant Subset scope achieves the highest RC (59.3%), a directional +4.0pp over Full schema (55.3%). The paired difference is not statistically significant after correction (Table 6), so we do not claim schema scoping alone is a decisive accuracy intervention on this four-table benchmark. It remains operationally useful because it reduces prompt size from 3,739 to 2,341 average input tokens while preserving or slightly improving correctness. Progressive scope remains the weakest strategy (43.3% RC), suggesting that incremental schema presentation can disrupt holistic database understanding.

6.3 RQ3: Metadata Enrichment

Building on Markdown format and Relevant Subset scope, we evaluate five metadata enrichment levels (Table 4).

Descriptions and Statistics both achieve 60.7% RC, a modest directional +1.4pp over the no-metadata baseline. Sample Values and the combined "All" configuration match baseline performance (59.3%). Because these differences are small and non-significant, we interpret metadata as a low-risk enrichment rather than a primary source of accuracy improvement.

Table 4: Metadata Enrichment Comparison. RC values show 95% bootstrap CIs.

Metadata	RC
Descriptions	0.607 (0.53–0.69)
Statistics	0.607 (0.53–0.69)
None	0.593 (0.51–0.67)
Sample Values	0.593 (0.51–0.67)
All	0.593 (0.51–0.67)

Table 5: Example Strategy Comparison. RC values show 95% bootstrap CIs.

Strategy	RC
Dynamic Few-Shot	0.660 (0.59–0.74)
Schema-Matched	0.620 (0.54–0.69)
Static Few-Shot	0.607 (0.53–0.69)
Zero-Shot	0.607 (0.53–0.69)

Table 6: Planned headline contrasts on the 150-query Custom Analytics benchmark. Values report result correctness; p-values use paired McNemar tests. Holm correction is across these five planned contrasts.

Contrast	ΔRC	p	p_H
Relevant subset vs. full schema	+4.0pp	0.308	0.615
Descriptions vs. no metadata	+1.3pp	0.774	0.774
Dynamic vs. zero-shot	+5.3pp	0.152	0.606
Dynamic vs. static few-shot	+5.3pp	0.152	0.606
Best config vs. full-schema zero-shot	+10.7pp	0.015	0.075

6.4 RQ4: Example Selection Strategy

Using the best format, scope, and metadata from preceding dimensions, we evaluate four example selection strategies (Table 5).

Dynamic few-shot achieves the best RC (66.0%), a directional +5.3pp over zero-shot and static few-shot. This isolated example-selection effect is not significant after Holm correction, but the full prompt configuration is meaningfully better than the full-schema zero-shot baseline in the planned paired contrast (+10.7pp, raw p=0.015; Holm-adjusted p=0.075). Schema-matched examples achieve 62.0% RC (+1.3pp over zero-shot). We therefore treat dynamic examples as part of the best system configuration, not as a standalone statistically decisive factor.

The few-shot audit found no exact natural-language overlap between the 44-example pool and the evaluation set. It flagged one exact SQL overlap and three high-similarity question pairs (Jaccard ≥ 0.8). Excluding these three queries leaves the dynamic few-shot result unchanged at 66.0% RC (97/147), so the best-configuration result is not driven by near-duplicate examples.

The best configuration performs best on Simple SELECT (20/25 correct, 80.0%) and Aggregation (23/30, 76.7%) queries. Window Functions (14/25, 56.0%) and Complex JOINs (10/20, 50.0%) remain the most challenging categories.

As a system migration, the final configuration replaces DataPup's current full-schema JSON prompt style. Under the same controlled

Table 7: System Prompt Component Ablation (Additive). Starting from a minimal prompt, each row adds one component. ΔRC is the marginal improvement from each addition.

Configuration	EX	RC	ΔRC
Minimal (base instructions only)	0.860	0.527	—
+ ClickHouse Dialect hints	0.907	0.573	+4.6pp
+ JOIN guidance	0.947	0.620	+4.7pp
+ Window function guidance	1.000	0.687	+6.7pp
+ Function reference (Full V6)	0.980	0.640	−4.7pp

Table 8: Cross-Model Comparison on Key Configurations. RC values show 95% bootstrap CIs.

Configuration	Sonnet 3.5	Sonnet 4
Baseline (Md., Full, None, Zero)	0.553 (0.47–0.63)	0.573 (0.49–0.65)
+ Relevant Subset	0.593 (0.51–0.67)	0.593 (0.51–0.67)
+ Descriptions	0.607 (0.53–0.69)	—
+ Dynamic Few-Shot (Best)	0.660 (0.59–0.74)	0.680 (0.61–0.75)

Claude 3.5 setting, that migration raises EX from 48.7% to 97.3% and RC from 17.3% (26/150) to 66.0% (99/150). The paired result is decisive: 76 queries are correct only under the revised prompt, compared with 3 correct only under the current-style prompt (McNemar exact p=2.7×10^{-19}).

6.5 RQ5: System Prompt Ablation

Our system prompt includes several dialect-specific guidance components. To quantify each component's marginal contribution, we conduct an additive ablation study (Table 7), starting from a minimal prompt and incrementally adding components.

The ablation reveals that window function and aggregation guidance contributes the most to system prompt effectiveness (+6.7pp, 41.9% of total improvement), followed by JOIN guidance (+4.7pp, 29.4%) and ClickHouse dialect hints (+4.6pp, 28.7%). Notably, adding the full function reference *hurts* performance by −4.7pp from the peak, suggesting that excessive reference material causes information overload. The total improvement from minimal to best (with window guidance) is +16.0pp RC (52.7% → 68.7%), suggesting that domain-specific guidance is a substantial contributor to overall accuracy.

6.6 RQ6: Cross-Model Validation

To assess whether our prompt engineering findings generalize beyond a single model, we evaluate key configurations on both Claude 3.5 Sonnet and Claude Sonnet 4 (Table 8).

Claude Sonnet 4 results are consistent with Claude 3.5 Sonnet findings. The best configuration achieves 68.0% RC on Sonnet 4 compared to 66.0% on Sonnet 3.5, and the baseline achieves 57.3% on both models. The relative rankings of configurations hold across model generations: relevant-subset scoping improves over the full-schema baseline, and dynamic few-shot examples are part of the strongest configuration. The magnitude of improvement from baseline to best is similar (+10.7pp for Sonnet 4 vs. +10.7pp for Sonnet 3.5), suggesting within-family robustness.

6.7 RQ7: Cross-Provider CLI Validation

The controlled ablation establishes the prompt-design effect under stable API conditions. For a DASHSys setting, however, the deployed concern is broader: users often combine database clients with heterogeneous coding assistants, local CLIs, and long-running shell workflows. We therefore reran the baseline and best configurations through Claude, Codex, and Gemini CLIs on the full 150-query custom analytics benchmark. Table 9 reports model success, execution success, and result correctness before and after one targeted execution-repair attempt.

The best configuration remains strongest for every provider. After repair, it improves over the corresponding baseline by +5.3pp for Claude, +5.4pp for Codex, and +13.3pp for Gemini. This is lower than the controlled Claude API ablation peak, which is expected: CLI tools introduce uncontrolled model versions, timeout behavior, and wrapper overhead. The important operational finding is that the direction of the prompt effect survives across providers. The same schema and example design also reduces Gemini's timeout sensitivity: Gemini best produces SQL for 94.0% of queries, compared with 88.7% for the baseline.

6.8 RQ8: Focused DuckDB Cross-Dialect Validation

To address whether the DataPup prompt design is tied only to ClickHouse, we built a local DuckDB copy of the custom analytics database and translated the benchmark gold SQL where the semantics were portable. This yields 130/150 executable DuckDB queries; the 20 excluded queries primarily exercise ClickHouse-only array/map functions such as `arrayJoin`, `groupArray`, and `quantiles`. We then reran Claude CLI generation with a DuckDB schema and dialect prompt.

The revised prompt matches the full-schema baseline on RC while eliminating execution failures (100.0% vs. 97.7% EX) and improving partial result score (0.635 vs. 0.622). This is not evidence that all ClickHouse-specific conclusions transfer unchanged; rather, it shows that the schema-presentation and dialect-guidance approach can be instantiated for a second analytical engine without losing correctness on portable analytical tasks.

6.9 RQ9: Execution Repair and Failure Modes

Execution repair is the mechanism a human-facing database client can use to turn syntactic failures into recoverable interactions. In our targeted repair experiment, the system executes the generated SQL, sends only failed-execution SQL back to the same CLI with the ClickHouse error message, and accepts the first repaired query that executes. This changes the operational profile more than the semantic ceiling: most syntax and type errors become executable, but wrong granularity, wrong filters, and wrong result shape remain.

Table 11 explains why repair is necessary but insufficient. Across the best configurations, repair lifts final execution success to 99.3% for Claude, 99.3% for Codex, and 93.3% for Gemini, yet result correctness remains between 50.7% and 56.0%. The remaining errors are predominantly semantic: the query executes but answers a related question at the wrong aggregation level, includes or omits an output column, chooses a different ranking/tie-breaking strategy, or silently changes grouping and filter assumptions. An automatic

Table 9: Cross-provider CLI validation on the 150-query Custom Analytics benchmark. "Gen." columns score the first generated SQL. "Repair" columns rescore after one repair attempt for failed-execution SQL only.

CLI	Config	MS	Gen. EX	Gen. RC	Repair EX	Repair RC	RS
Claude	Baseline	100.0	94.0	48.0	100.0	50.7	9/9
Claude	Best	100.0	95.3	54.0	99.3	56.0	6/7
Codex	Baseline	100.0	98.0	44.7	100.0	45.3	3/3
Codex	Best	100.0	96.7	49.3	99.3	50.7	4/5
Gemini	Baseline	88.7	83.3	37.3	86.7	40.0	5/6
Gemini	Best	94.0	92.7	52.7	93.3	53.3	1/1

Table 10: Focused DuckDB validation on 130 portable custom analytics queries.

Config	Queries	EX	RC
Full JSON zero-shot	130	97.7%	60.8%
Revised prompt	130	100.0%	60.8%

Table 11: Dominant failure modes after one repair attempt for best configurations. Counts aggregate Claude, Codex, and Gemini best runs (450 total queries).

Outcome / Failure Mode	Count	Share
Correct	240	53.3%
Wrong granularity or filter	66	14.7%
Projection mismatch	48	10.7%
Wrong values or ranking	32	7.1%
Grouping-key mismatch	26	5.8%
Missing filter assumption	23	5.1%
Timeout / empty output	10	2.2%
Residual execution error	2	0.4%
Other wrong result	3	0.7%

audit of wrong-executable failures tags the most common assumptions that should be surfaced to analysts: ranking/tie-breaking (192 cases), metric definition (127), filters/predicates (120), time windows/buckets (97), and grouping keys (94). The audit uses deterministic keyword and structural checks over generated SQL and result metadata; tags are multi-label, so counts exceed the number of wrong executable failures. These are precisely the cases where an analyst-facing system should expose intermediate assumptions, preview result shape, and support conversational refinement rather than treating executable SQL as a completed task.

6.10 RQ10: Cross-Dataset Generalization

We evaluate the optimal custom-analytics configuration on ClickBench and SSB to measure transfer across schema structures. Cross-dataset results reveal a substantial transfer gap: ClickBench achieves only 2.3% RC (1/43) despite 83.7% EX, and SSB achieves 7.7% RC (1/13) with 38.5% EX. These low values reflect dataset-specific query patterns and examples drawn from the custom analytics pool. On SSB, the full-schema baseline (30.8% RC, 4/13) outperforms relevant-subset scoping (7.7%, 1/13) because the star schema requires all five tables. We therefore narrow our claims: the measured prompt design improves DataPup's analytics workload and can be instantiated for DuckDB on portable tasks, while cross-schema deployment requires dataset-specific schema selection and examples.

6.11 RQ11: External Baseline Comparison

To contextualize our results against existing Text-to-SQL methods, we adapt DAIL-SQL [6] for ClickHouse by using its question-skeleton similarity method for example selection with our prompt template. DAIL-SQL achieves 66.0% RC (99/150) with 98.7% EX, compared to our optimal configuration's 66.7% RC with 98.0% EX. The near-identical RC cautions against overclaiming the specific selector: the practical contribution is an audited, retrievable example pool inside a database client rather than a new example-selection algorithm.

7 DISCUSSION

7.1 Practical Recommendations

Based on our ablation analysis and experience building DataPup, we offer the following guidelines for practitioners building AI-assisted database clients targeting OLAP systems:

(1) **Use compact structured schema formats**—Markdown and DDL both work well; JSON and prose are risky for ClickHouse because they either dilute attention or omit exact identifiers.
(2) **Filter to relevant tables when schemas grow**—the accuracy gain is directional in our four-table workload, but token savings are substantial.
(3) **Treat column descriptions as low-risk metadata**—they show small directional gains, not a statistically decisive effect.
(4) **Audit few-shot retrieval pools**—dynamic examples help in the best configuration, but leakage and near-duplicates must be checked explicitly.
(5) **Invest in dialect-specific system prompts** with function references, JOIN hints, and syntax guidance.
(6) **Separate execution repair from semantic validation**—repair makes many invalid SQL strings executable, but does not guarantee user-intent correctness.
(7) **Avoid verbose formats** (JSON) and unstructured formats (natural language) for schema presentation.
(8) **Evaluate beyond execution accuracy**—the EX-RC gap can exceed 60pp, masking fundamental correctness failures.

7.2 DASHSys Perspective

The CLI validation and repair results shift DataPup from a pure Text-to-SQL benchmark to a human-facing data-system problem. A database client should help users detect plausible-but-wrong queries, recover from execution errors, and refine ambiguous analytical intent. Our failure taxonomy shows that after one repair attempt, syntactic failures nearly disappear for Claude and Codex, but semantic failures persist. DataPup should therefore surface assumptions such as filters, grouping keys, tie-breaking rules, and date windows, then make revision cheap.

7.3 Implications for DataPup

The research findings from this paper are being integrated back into DataPup's production pipeline (Section 2):

- **Schema format**: Migrating from raw JSON to Markdown schema presentation, which improves execution accuracy (+44.0pp over JSON) and readability.
- **Schema scope**: Implementing keyword-based relevant-subset table selection to reduce prompt size, with fallback to full schema for star schemas and JOIN-heavy workloads.
- **Metadata**: Adding column descriptions to the schema introspection output as a low-risk enrichment.
- **Examples**: Integrating audited dynamic few-shot example selection from a curated pool that is separate from evaluation targets.
- **Repair loop**: Incorporating execution-error repair as a separate stage from generation, with UI affordances that make remaining semantic assumptions visible to the analyst.

7.4 Threats to Validity

Internal validity. We use temperature 0 for deterministic output. Bootstrap confidence intervals (10,000 iterations) quantify sampling uncertainty over the query set. Our evaluation pipeline underwent iterative refinement (Section 4.5), and all reported results use the final pipeline version.

External validity. The primary controlled ablation targets ClickHouse. The DuckDB validation covers a second analytical engine, but only on the 130 portable custom-analytics queries and with one CLI model; it does not substitute for broader dialect replication. The ClickBench and SSB results also show that cross-schema transfer requires dataset-specific schema selection and examples.

Construct validity. Our RC metric uses tolerance-based numeric comparison (1% relative tolerance), percentage normalization, and order-insensitive matching. Some semantically equivalent results with different structures may still be scored as incorrect.

Model validity. The controlled ablation uses two models from the same family (Anthropic Claude). Cross-model validation between Claude 3.5 Sonnet and Claude Sonnet 4 addresses within-family robustness. The CLI experiment adds Claude, Codex, and Gemini command-line assistants, but those tools are black boxes with uncontrolled model versions, decoding settings, wrapper behavior, and timeout characteristics. We therefore use them as operational validation, not as a clean model leaderboard.

Human-loop validity. Our repair experiment measures an automated recovery step that would appear inside a human-facing client, but it is not a user study. The paper therefore supports claims about failure modes and system design requirements, not claims about analyst task-completion time or subjective usability.

8 RELATED WORK

Early benchmarks include WikiSQL [23]; later work added cross-database Spider [22], larger real-world settings such as BIRD [10], enterprise workflows in Spider 2.0 [9], and domain-specific scientific schemas [1]. Earlier model designs emphasized schema encoders and constrained decoding [2, 17, 19]. More recent LLM systems study demonstration selection, decomposition, zero-shot prompting, multi-agent collaboration, schema decomposition, and type enrichment [5–7, 11, 15, 18]. Our focus differs by studying a deployed analytical database client and the schema/execution loop that shapes its human-facing behavior.

Our prompt findings relate to chain-of-thought, self-consistency, few-shot learning, and broader prompting work [3, 12, 13, 20, 21]. In this domain, general-purpose reasoning techniques do not uniformly improve SQL generation once the prompt already includes dialect and schema guidance. Methodologically, we follow work emphasizing execution-based SQL evaluation [8, 10, 16], but show that execution accuracy can mask semantic failures. This motivates result correctness, repair analysis, and human-visible assumptions as first-class evaluation concerns for data-centric agents.

9 CONCLUSION

This paper presents DataPup as a human-facing analytical database client and evaluates schema-aware prompting choices that affect its SQL generation loop. The revised prompt raises DataPup's current-style JSON/full-schema RC from 17.3% to 66.0%, and reaches 66.0% versus 55.3% for the stronger Markdown full-schema zero-shot baseline; this +10.7pp planned contrast has raw p=0.015 and is marginal after Holm correction (p_H=0.075). CLI validation across Claude, Codex, and Gemini confirms the best configuration remains strongest after one repair attempt, while DuckDB validation shows the approach can be instantiated for a second analytical engine on portable tasks. Failure analysis shows that repair removes many syntax errors without solving semantic ambiguity. These findings are being folded into DataPup through compact schema presentation, audited example retrieval, targeted execution repair, and UI affordances that expose query assumptions to analysts. Future work includes broader dialect replications, learned schema relevance, and user studies; we release the benchmark, evaluation framework, and artifacts.[2]

REFERENCES

[1] Bayer, T., Brendel, A., et al. ScienceBenchmark: A Complex Real-World Benchmark for Evaluating Natural Language to SQL Systems. *PVLDB*, 17(4):685–698, 2024.

[2] Bogin, B., Berant, J., Gardner, M. Representing Schema Structure with Graph Neural Networks for Text-to-SQL Parsing. In *Proc. ACL*, pp. 4560–4565, 2019.

[3] Brown, T., Mann, B., Ryder, N., et al. Language Models are Few-Shot Learners. In *Advances in NeurIPS*, vol. 33, 2020.

[4] ClickHouse, Inc. ClickBench: a Benchmark For Analytical DBMS. https://benchmark.clickhouse.com/, 2023.

[5] Dong, X., Zhang, C., Ge, Y., et al. C3: Zero-shot Text-to-SQL with ChatGPT. *arXiv:2307.07306*, 2023.

[6] Gao, D., Wang, H., Li, Y., et al. Text-to-SQL Empowered by Large Language Models: A Benchmark Evaluation. *PVLDB*, 17(5):1132–1145, 2024.

[2] https://github.com/DataPupOrg/DataPup-Research

[7] Guo, C., Tian, Z., Tang, J., et al. Chase: A Large-Scale and Pragmatic Chinese Dataset for Cross-Database Context-Dependent Text-to-SQL. In *Proc. ACL-IJCNLP*, pp. 1561–1570, 2021.

[8] Katsogiannis-Meimarakis, G., Koutrika, G. A Survey on Deep Learning Approaches for Text-to-SQL. *The VLDB Journal*, 32(4):905–936, 2023.

[9] Lei, F., Chen, J., Ye, Y., et al. Spider 2.0: Evaluating Language Models on Real-World Enterprise Text-to-SQL Workflows. *arXiv:2411.07763*, 2024.

[10] Li, J., Hui, B., Qu, G., et al. Can LLM Already Serve as A Database Interface? A BIg Bench for Large-Scale Database Grounded Text-to-SQL. In *Advances in NeurIPS*, vol. 36, 2023.

[11] Li, H., Zhang, J., Li, C., Chen, H. RESDSQL: Decoupling Schema Linking and Skeleton Parsing for Text-to-SQL. In *Proc. AAAI*, pp. 13174–13182, 2023.

[12] Liu, J., Shen, D., Zhang, Y., et al. What Makes Good In-Context Examples for GPT-3? In *Proc. DeeLIO Workshop, ACL*, pp. 100–114, 2022.

[13] Liu, P., Yuan, W., Fu, J., et al. Pre-train, Prompt, and Predict: A Systematic Survey of Prompting Methods in NLP. *ACM Computing Surveys*, 55(9):1–35, 2023.

[14] O'Neil, P., O'Neil, E., Chen, X., Revilak, S. The Star Schema Benchmark and Augmented Fact Table Indexing. In *TPCTC*, LNCS 5895, pp. 237–252. Springer, 2009.

[15] Pourreza, M., Rafiei, D. DIN-SQL: Decomposed In-Context Learning of Text-to-SQL with Self-Correction. In *Advances in NeurIPS*, vol. 36, 2023.

[16] Rajkumar, N., Li, R., Baber, D. Evaluating the Text-to-SQL Capabilities of Large Language Models. *arXiv:2204.00498*, 2022.

[17] Scholak, T., Schucher, N., Bahdanau, D. PICARD: Parsing Incrementally for Constrained Auto-Regressive Decoding from Language Models. In *Proc. EMNLP*, pp. 9895–9901, 2021.

[18] Wang, B., Ren, C., Yang, J., et al. MAC-SQL: A Multi-Agent Collaborative Framework for Text-to-SQL. In *Proc. COLING*, pp. 540–557, 2025.

[19] Wang, B., Shin, R., Liu, X., et al. RAT-SQL: Relation-Aware Schema Encoding and Linking for Text-to-SQL Parsers. In *Proc. ACL*, pp. 7567–7578, 2020.

[20] Wang, X., Wei, J., Schuurmans, D., et al. Self-Consistency Improves Chain of Thought Reasoning in Language Models. In *Proc. ICLR*, 2023.

[21] Wei, J., Wang, X., Schuurmans, D., et al. Chain-of-Thought Prompting Elicits Reasoning in Large Language Models. In *Advances in NeurIPS*, vol. 35, 2022.

[22] Yu, T., Zhang, R., Yang, K., et al. Spider: A Large-Scale Human-Labeled Dataset for Complex and Cross-Domain Semantic Parsing and Text-to-SQL Task. In *Proc. EMNLP*, pp. 3911–3921, 2018.

[23] Zhong, V., Xiong, C., Socher, R. Seq2SQL: Generating Structured Queries from Natural Language using Reinforcement Learning. *arXiv:1709.00103*, 2017.